"Develop a passion for learning."

MODULE 18

Mockito for Test Isolation

Learn how to use Mockito to create mocks, isolate dependencies, and test business logic with confidence.


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT

MODULE 18 OVERVIEW

Learn how to use Mockito to create mocks, isolate dependencies, and test business logic in Java applications.

- Real code depends on databases, external services, and other classes. Mockito lets you replace those dependencies with controllable test doubles, so you can test business logic in complete isolation.

DURATION & LAB

1 Hour Theory

Mockito architecture, mocking, stubbing, verification.

2 Hours Lab

Mockito and Mocking.

Scenario

Test DefaultCustomerService in isolation by mocking CustomerRepository.

FOCUS

Test Isolation, Reliability, Maintainability

CORE SKILLS

Create mocks, stub methods, verify interactions

THE PAYOFF

Fast, deterministic, dependency-free unit tests

KEY TAKEAWAY Mockito lets you test business logic in isolation — no database, no external services, just your code under test.

WHY MOCKING MATTERS

Testing against real dependencies makes tests slow, flaky, and hard to control.

WITHOUT MOCKING

- Tests hit a real database or external service
- Slow, flaky, and order-dependent tests
- Hard to simulate error scenarios
- Failures may be caused by infrastructure, not code

VS

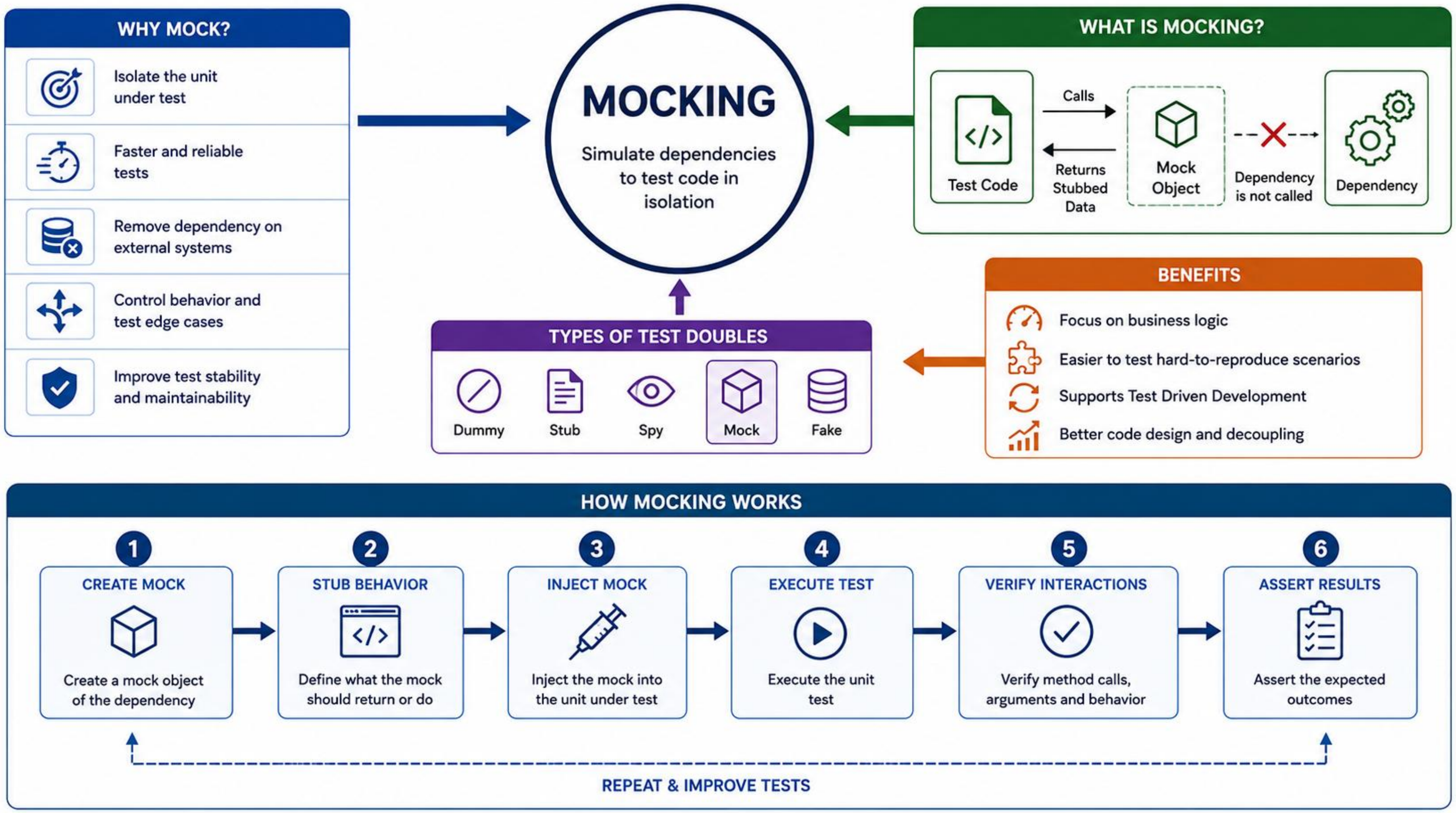
WITH MOCKING

- Dependencies replaced with controllable test doubles
- Fast, deterministic, isolated tests
- Easy to simulate success and failure scenarios
- Failures point directly to the unit under test

MOCKING ALLOWS US TO

- Speed up tests — no real calls to databases, APIs, or external systems.
- Improve reliability — tests don't fail due to external issues outside your control.
- Gain control over dependencies — define exactly what a dependency returns or does.
- Test edge cases easily — simulate exceptions, timeouts, and unusual scenarios.
- Encourage clean, loosely coupled code that's more maintainable and testable.

KEY TAKEAWAY Mocking isolates the unit under test so failures point to real bugs, not flaky infrastructure.



WHY MOCK?
Isolate the unit under test
Faster and reliable tests
Remove dependency on external systems
Control behavior and test edge cases
Improve test stability and maintainability
MOCKING
Simulate dependencies to test code in isolation
WHAT IS MOCKING?
Test Code
Calls
Returns Stubbed Data
Mock Object
Dependency is not called
Dependency
TYPES OF TEST DOUBLES
Dummy
Stub
Spy
Mock
Fake
BENEFITS
Focus on business logic
Easier to test hard-to-reproduce scenarios
Supports Test Driven Development
Better code design and decoupling
HOW MOCKING WORKS
1
CREATE MOCK
Create a mock object of the dependency
2
STUB BEHAVIOR
Define what the mock should return or do
3
INJECT MOCK
Inject the mock into the unit under test
4
EXECUTE TEST
Execute the unit test
5
VERIFY INTERACTIONS
Verify method calls, arguments and behavior
6
ASSERT RESULTS
Assert the expected outcomes
REPEAT & IMPROVE TESTS

TEST DOUBLES OVERVIEW

"Test double" is the umbrella term for objects that stand in for real dependencies during testing.

TYPE	PURPOSE	EXAMPLE
`Dummy`	Passed but never actually used	A placeholder object required by a method signature
`Stub`	Returns predefined answers to calls	when(repo.findById(1L)).thenReturn(user)
`Mock`	Stub + records interactions for verification	verify(repo).save(any(User.class))
`Spy`	Wraps a real object, allows partial stubbing	spy(new OrderService())
`Fake`	A working, simplified implementation	An in-memory repository instead of a DB

Purpose: replace real collaborators with controlled objects so you can test one unit of code in isolation.
Note: These categories describe roles, not necessarily different libraries or object types. A Mockito mock may be used only as a stub in one test. A fake is usually a working implementation, not created by Mockito. A spy wraps or partially delegates to a real object. A mock is most useful when interaction verification matters.
Takeaway: Choose the simplest test double that gives the test the control it needs. Do not default to a mock when a real value object or small fake is clearer.

KEY TAKEAWAY Mockito creates configurable test doubles that can be used for stubbing, interaction verification, or partial delegation.

TEST DOUBLES

Test doubles are stand-ins for real dependencies used in tests.

DUMMY

Passed around but never actually used.

Just to satisfy parameters.

STUB

Provides predefined answers to method calls.

Used to control outputs.

SPY

Wraps real object and records interactions.

Can verify calls and use real behavior.

MOCK

Expects specific interactions and can be programmed.

Verifies behavior and controls outputs.

FAKE

Working implementation with simplified logic.

Used instead of real dependency.

SCOPE OF USE

- Filling parameters not used
- Returning controlled data / values
- Partial mocking and interaction testing
- Behavior verification and expectations
- Replacing complex or external systems

TYPE	DUMMY	STUB	SPY	MOCK	FAKE
PURPOSE	Just to pass objects	Provide test data	Observe interactions	Verify interactions	Work like real but simpler
BEHAVIOR	No behavior	Returns programmed values	Calls real methods (can stub some)	Does not call real methods (unless stubbed)	Real implementation (simplified)
VERIFIES	✗ No	✗ No	✓ Yes	✓ Yes	✗ No
EXAMPLE USE	Method parameter not used	Repository returning user data	Service method calls and partial mocks	Verify repository.save() was called	In-memory database, fake email service

HOW MOCKITO WORKS

Mockito's core components work together in a repeatable cycle — from creating mocks to verifying interactions.

1 **Mock Creation** → 2 **Stubbing Engine** → 3 **Verification Engine** → 4 **Mock Proxy**

1 Mock Creation

Creates mock objects using Byte Buddy.

2 Stubbing Engine

Records stub definitions and returns values.

3 Verification Engine

Captures and verifies interactions.

4 Mock Proxy

Runtime-generated test double, commonly backed by Byte Buddy.

HOW MOCKITO WORKS — CORRECT STEP ORDER

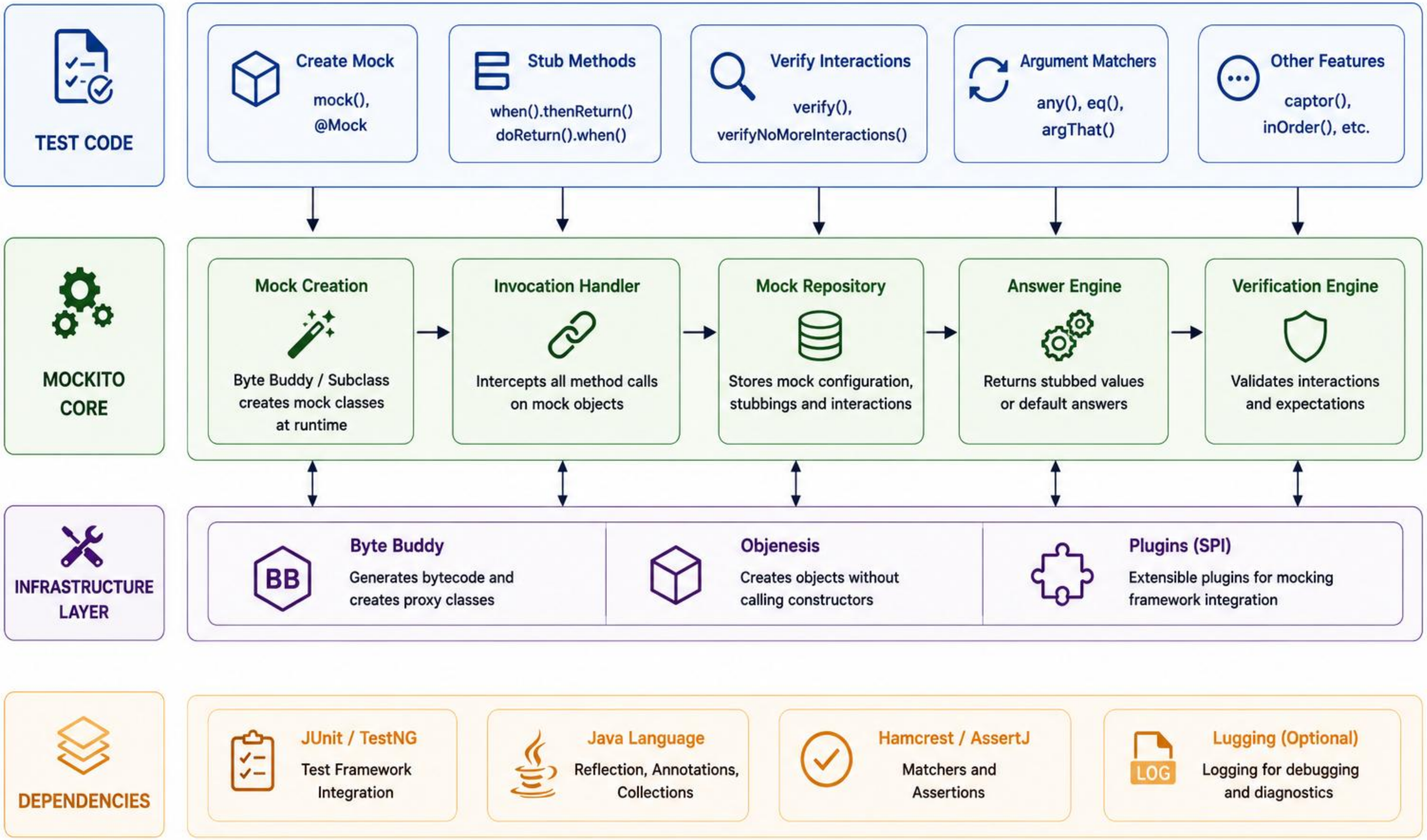
1. Create mocks
2. Stub required behavior
3. Invoke the class under test
4. Assert outcome
5. Verify important interactions

Simple diagram: Test → Real Service → Mock Repository

Mockito generates runtime test doubles using its mock maker, commonly backed by Byte Buddy. The exact mechanism depends on the mocked type and configuration.

KEY TAKEAWAY Every Mockito test follows the same cycle: create mocks, stub required behavior, invoke the class under test, assert the outcome, then verify important interactions — stubbing always comes before invocation.

MOCKITO ARCHITECTURE



CREATING MOCKS

Mockito provides several ways to create mock objects for your dependencies.

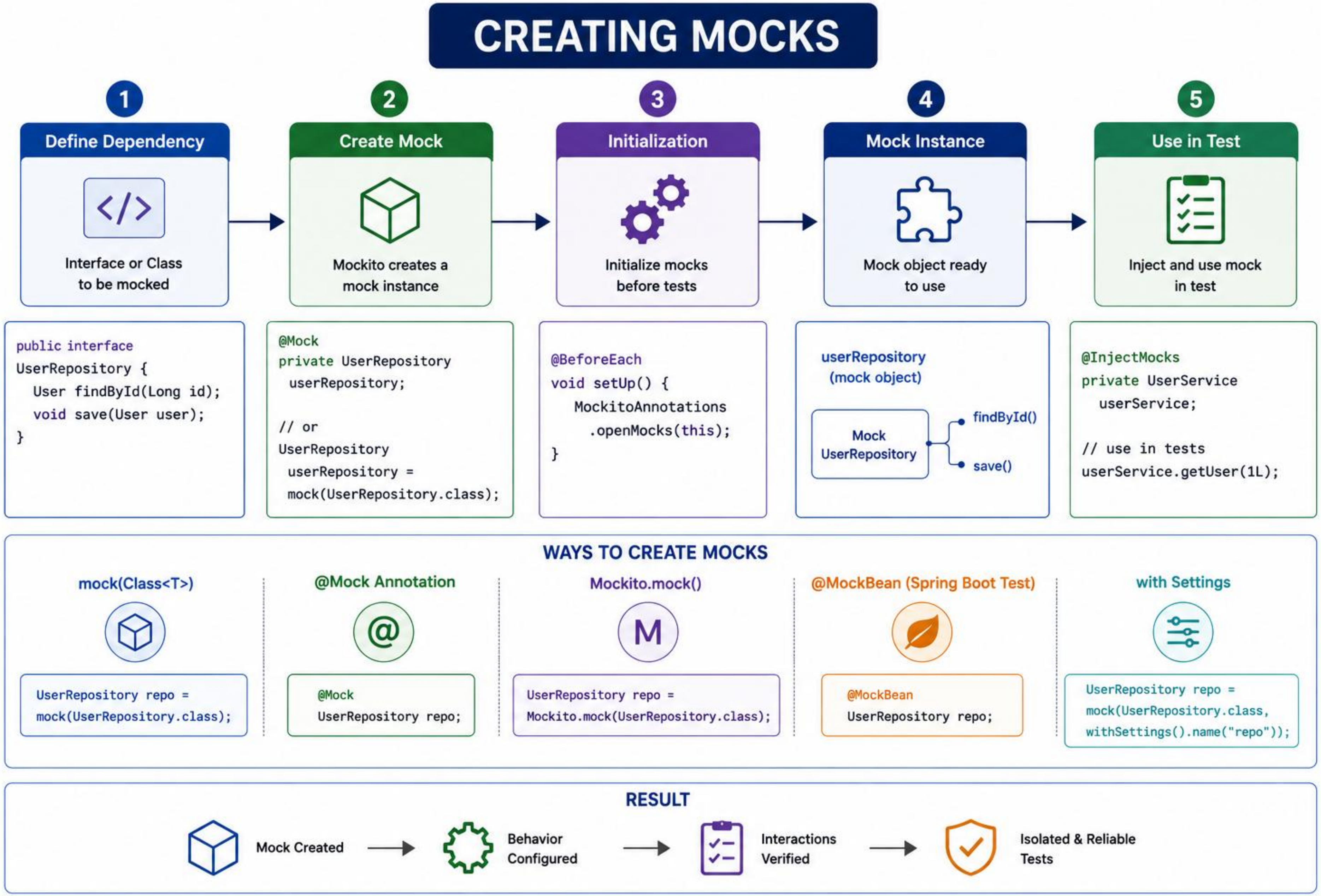
EXAMPLE

```
@ExtendWith(MockitoExtension.class)
class DefaultCustomerServiceTest {
  @Mock
  CustomerRepository repository;
  CustomerValidator validator =
      new CustomerValidator();
  DefaultCustomerService service;
  @BeforeEach
  void setUp() {
    service = new DefaultCustomerService(
        repository, validator);
  }
}
```

KEY POINTS

- @Mock creates a mock object for a class or interface.
- Prefer explicit construction in @BeforeEach — pass mocks and real collaborators into the constructor directly for clarity.
- @InjectMocks is a convenience alternative: it creates the object under test and injects matching mocks automatically.
- @ExtendWith(MockitoExtension.class) is the simplest, primary way to enable annotations in JUnit 5.
- Alternative: Mockito.mock(Class) directly, or MockitoAnnotations.openMocks(this) — if used, close the returned AutoCloseable.
- Advanced: use MockSettings for advanced mock configuration (e.g., extra interfaces, custom answers).
- Convenience snippet: @InjectMocks DefaultCustomerService service; — declared once, wired automatically once all constructor dependencies are mocked or set.

KEY TAKEAWAY Mock dependencies whose real behavior would make the test slow, nondeterministic, difficult to control, or outside the unit's responsibility. Use real objects for simple, deterministic collaborators where that improves clarity.



CREATING MOCKS
1
Define Dependency
Interface or Class to be mocked
public interface UserRepository {
User findById(Long id);
void save(User user);
}
2
Create Mock
Mockito creates a mock instance
@Mock
private UserRepository userRepository;
// or
UserRepository userRepository = mock(UserRepository.class);
3
Initialization
Initialize mocks before tests
@BeforeEach
void setUp() {
MockitoAnnotations.openMocks(this);
}
4
Mock Instance
Mock object ready to use
userRepository (mock object)
Mock UserRepository
findById()
save()
5
Use in Test
Inject and use mock in test
@InjectMocks
private UserService userService;
// use in tests
userService.getUser(1L);
WAYS TO CREATE MOCKS
mock(Class<T>)
UserRepository repo = mock(UserRepository.class);
@Mock Annotation
@Mock UserRepository repo;
Mockito.mock()
UserRepository repo = Mockito.mock(UserRepository.class);
@MockBean (Spring Boot Test)
@MockBean UserRepository repo;
with Settings
UserRepository repo = mock(UserRepository.class, withSettings().name("repo"));
RESULT
Mock Created
Behavior Configured
Interactions Verified
Isolated & Reliable Tests

STUBBING METHODS

Stubbing defines what a mock should return (or throw) when a method is called.

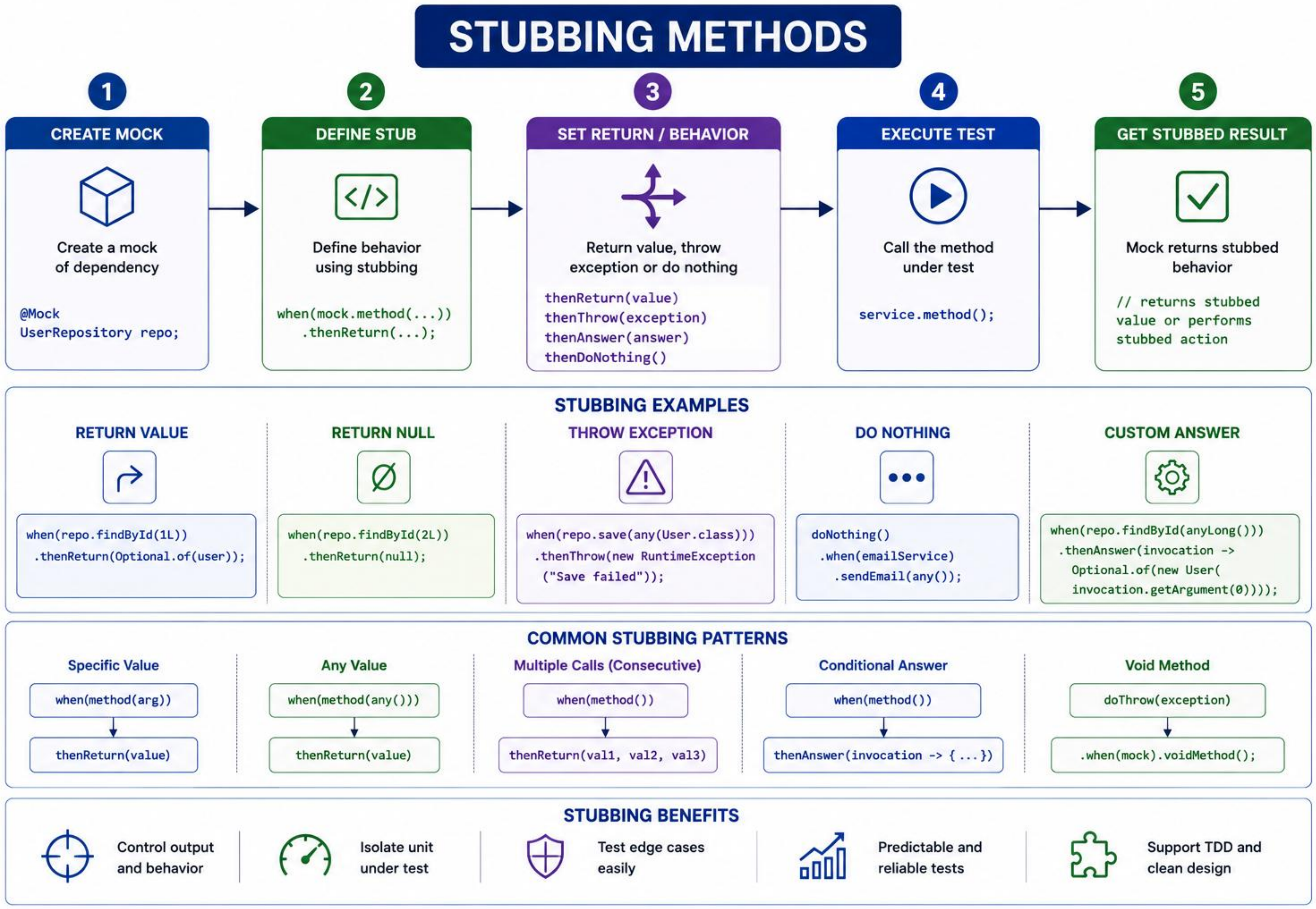
EXAMPLE

```
when(orderRepository.findById(1L))
    .thenReturn(Optional.of(order));
when(paymentGateway.charge(any()))
    .thenThrow(new PaymentException());

doReturn(value).when(spy).method();
doThrow(new NotificationException())
    .when(notificationService).send(any());

when(repository.findByStatusAndRegion(
    eq(ACTIVE), anyString()))
    .thenReturn(customers);
```

COMMON STUBBING METHODS

- when(mock.method()).thenReturn(value) — normal flow.
- when(mock.method()).thenThrow(exception) — exceptional flow.
- For void methods, use doNothing(), doThrow(), or doAnswer() — never doReturn().
- doReturn() is commonly useful for spies, or when when() would invoke real behavior.
- Use doNothing() only when it improves readability or is part of consecutive behavior. For ordinary mock void methods, no stubbing is required.
- Stub only what is necessary for the test.
- Advanced: thenReturn(v1, v2, v3) returns different values per call — can couple a test to invocation order.
- When one argument uses a matcher, all arguments in that invocation should normally use matchers.

KEY TAKEAWAY Stub only the behavior your test actually needs — over-stubbing makes tests brittle and hard to maintain.


STUBBING METHODS
1
CREATE MOCK
Create a mock of dependency
@Mock
UserRepository repo;
2
DEFINE STUB
Define behavior using stubbing
when(mock.method(...))
.thenReturn(...);
3
SET RETURN / BEHAVIOR
Return value, throw exception or do nothing
thenReturn(value)
thenThrow(exception)
thenAnswer(answer)
thenDoNothing()
4
EXECUTE TEST
Call the method under test
service.method();
5
GET STUBBED RESULT
Mock returns stubbed behavior
// returns stubbed value or performs stubbed action
STUBBING EXAMPLES
RETURN VALUE
when(repo.findById(1L))
.thenReturn(Optional.of(user));
RETURN NULL
when(repo.findById(2L))
.thenReturn(null);
THROW EXCEPTION
when(repo.save(any(User.class)))
.thenThrow(new RuntimeException
("Save failed"));
DO NOTHING
doNothing()
.when(emailService)
.sendEmail(any());
CUSTOM ANSWER
when(repo.findById(anyLong()))
.thenAnswer(invocation ->
Optional.of(new User(
invocation.getArgument(0))));
COMMON STUBBING PATTERNS
Specific Value
when(method(arg))
thenReturn(value)
Any Value
when(method(any()))
thenReturn(value)
Multiple Calls (Consecutive)
when(method())
thenReturn(val1, val2, val3)
Conditional Answer
when(method())
thenAnswer(invocation -> {...})
Void Method
doThrow(exception)
.when(mock).voidMethod();
STUBBING BENEFITS
Control output and behavior
Isolate unit under test
Test edge cases easily
Predictable and reliable tests
Support TDD and clean design

VERIFYING INTERACTIONS

Verification confirms that the unit under test called its dependencies as expected.

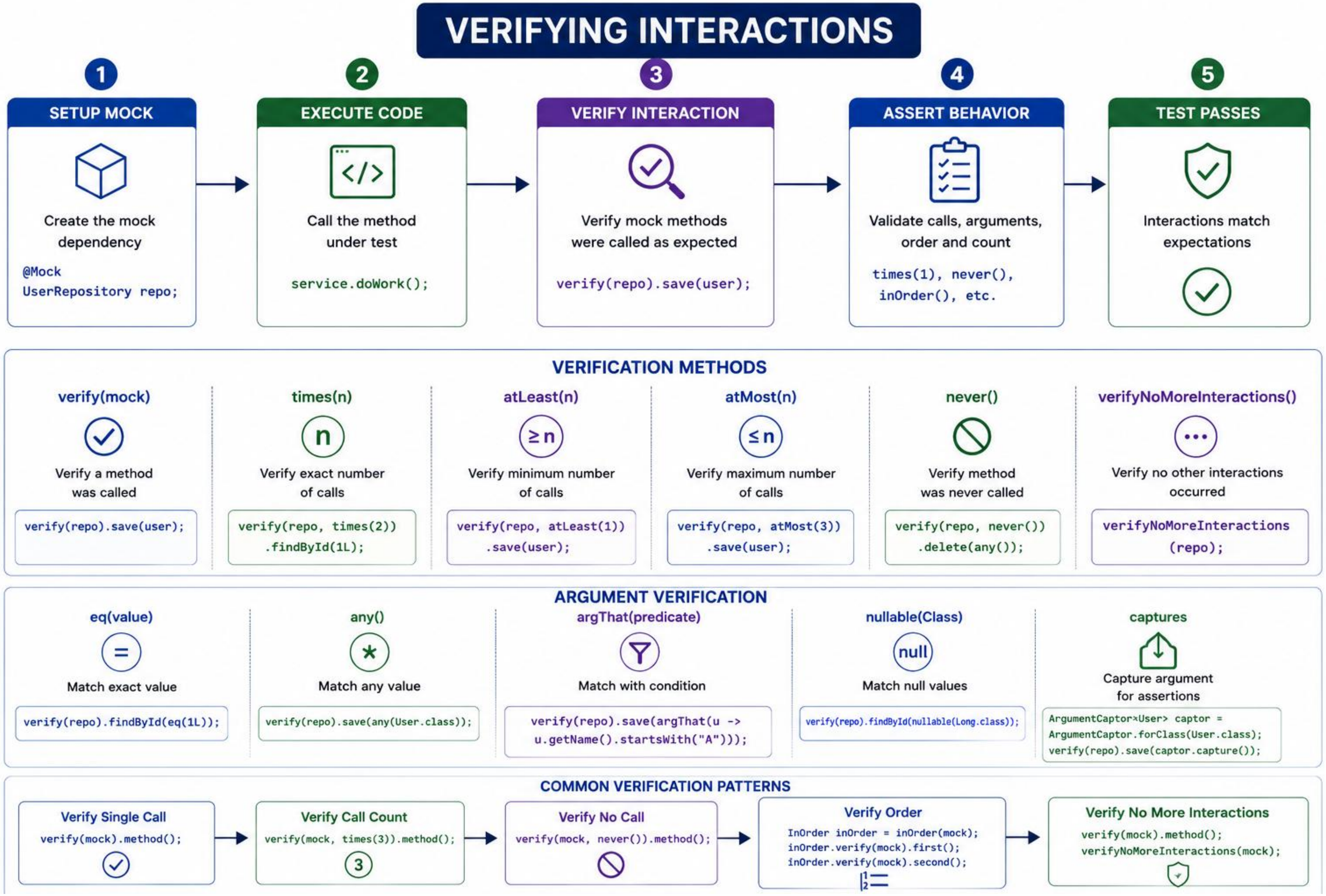
METHOD	PURPOSE	EXAMPLE
`verify(mock).method(args)`	Confirms method was called once with args	verify(repo).save(user)
`verify(mock, times(n))`	Confirms method was called exactly n times	verify(repo, times(2)).save(any())
`verify(mock, never())`	Confirms method was never called	verify(gateway, never()).charge(any())
ADVANCED VERIFICATION		
`verify(mock, atLeastOnce())`	Confirms method was called at least once	verify(notificationService).sendCustomerActivated(customerId);
`inOrder(...).verify(...)`	Confirms methods were called in sequence	inOrder.verify(repo).findById(1L)
`verifyNoMoreInteractions(mock)`	Confirms no other interactions occurred	verifyNoMoreInteractions(userRepository)

```
ArgumentCaptor<Customer> captor =
    ArgumentCaptor.forClass(Customer.class);
verify(repository).save(captor.capture());
Customer saved = captor.getValue();
assertEquals(ACTIVE, saved.status());
```

Captor: Use a captor when the argument itself is an important observable outcome. Do not use it when a direct argument equality check is enough.

Results first: Assert the observable result first. Verify interactions only when they are part of the expected behavior. Often: result, exception, entity state, captured object, side effect.
atLeastOnce(): Prefer exact verification when the call count is part of the behavior. Use atLeastOnce() only when additional calls are intentionally acceptable.
verifyNoMoreInteractions(): Use it only when interaction completeness is part of the contract, such as security-sensitive side effects or ensuring no persistence occurs after rejection.

KEY TAKEAWAY Verify behavior, not implementation — confirm the important interactions happened, not every internal detail.


VERIFYING INTERACTIONS
1
SETUP MOCK
Create the mock dependency
@Mock UserRepository repo;
2
EXECUTE CODE
Call the method under test
service.doWork();
3
VERIFY INTERACTION
Verify mock methods were called as expected
verify(repo).save(user);
4
ASSERT BEHAVIOR
Validate calls, arguments, order and count
times(1), never(), inOrder(), etc.
5
TEST PASSES
Interactions match expectations
VERIFICATION METHODS
verify(mock)
Verify a method was called
verify(repo).save(user);
times(n)
n
Verify exact number of calls
verify(repo, times(2)) .findById(1L);
atLeast(n)
≥ n
Verify minimum number of calls
verify(repo, atLeast(1)) .save(user);
atMost(n)
≤ n
Verify maximum number of calls
verify(repo, atMost(3)) .save(user);
never()
Verify method was never called
verify(repo, never()) .delete(any());
verifyNoMoreInteractions()
Verify no other interactions occurred
verifyNoMoreInteractions (repo);
ARGUMENT VERIFICATION
eq(value)
=
Match exact value
verify(repo).findById(eq(1L));
any()
*
Match any value
verify(repo).save(any(User.class));
argThat(predicate)
Match with condition
verify(repo).save(argThat(u -> u.getName().startsWith("A")));
nullable(Class)
null
Match null values
verify(repo).findById(nullable(Long.class));
captures
Capture argument for assertions
ArgumentCaptor<User> captor = ArgumentCaptor.forClass(User.class); verify(repo).save(captor.capture());
COMMON VERIFICATION PATTERNS
Verify Single Call
verify(mock).method();
Verify Call Count
verify(mock, times(3)).method();
3
Verify No Call
verify(mock, never()).method();
Verify Order
InOrder inOrder = inOrder(mock); inOrder.verify(mock).first(); inOrder.verify(mock).second();
Verify No More Interactions
verify(mock).method(); verifyNoMoreInteractions(mock);

MOCKING REPOSITORIES

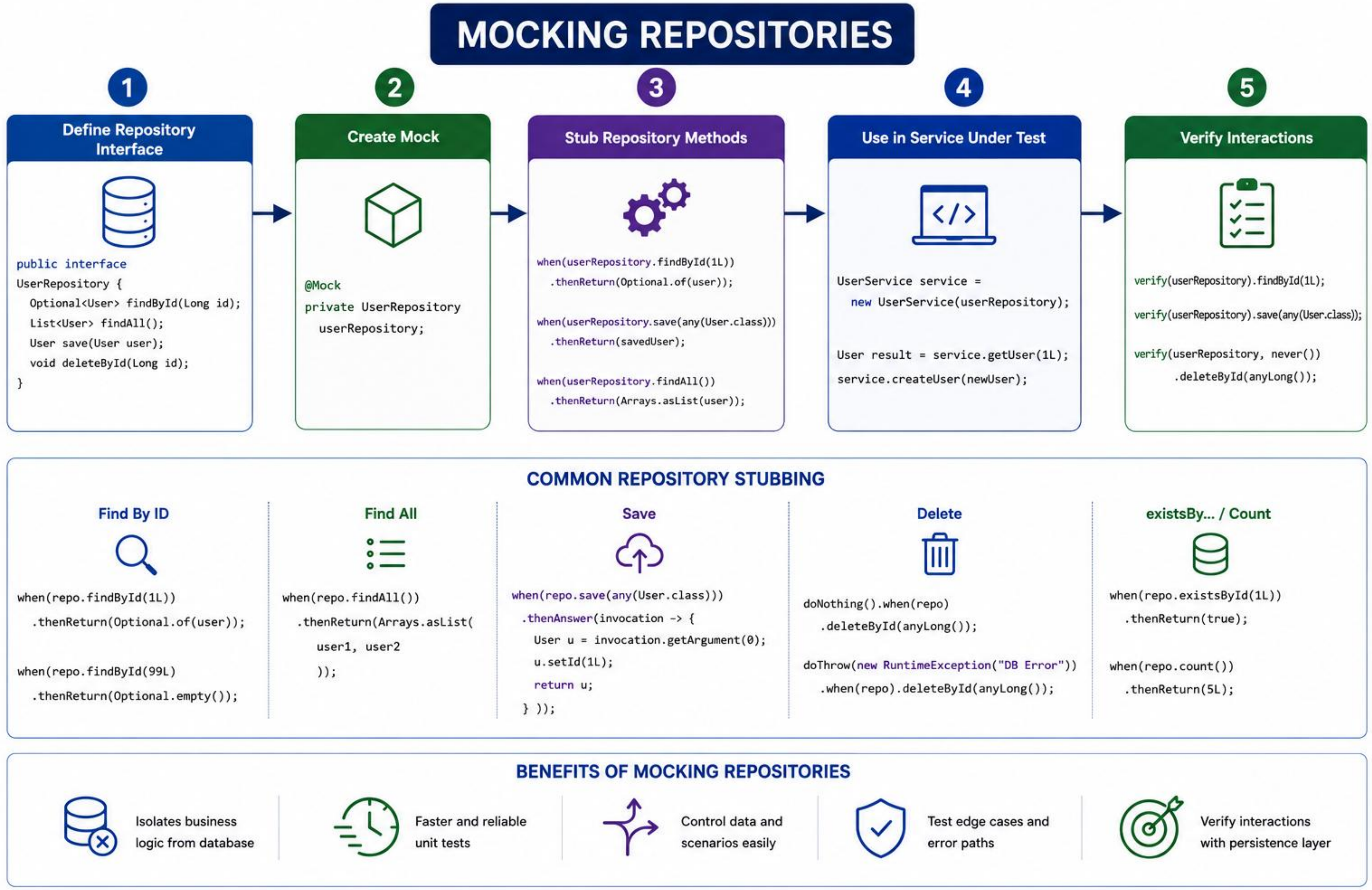
Repositories abstract data access — mock them to control responses and isolate business logic from the database.

EXAMPLE

```
@Test
void getCustomer_whenCustomerExists_returnsCustomer() {
  when(repository.findById("CUS-1001"))
      .thenReturn(Optional.of(customer));
  Customer result =
      service.getCustomer("CUS-1001");
  assertAll(
      () -> assertEquals("CUS-1001", result.id()),
      () -> assertEquals("Amina Khan", result.name())
  );
  verify(repository).findById("CUS-1001");
}
```

Note: Mockito verifies service behavior, not persistence integration. Repository behavior still requires integration tests against the real persistence technology.

KEY TAKEAWAY Prefer real domain objects and value objects. Mock infrastructure boundaries and collaborators outside the behavior being tested.


MOCKING REPOSITORIES
1
Define Repository Interface
public interface
UserRepository {
Optional<User> findById(Long id);
List<User> findAll();
User save(User user);
void deleteById(Long id);
}
2
Create Mock
@Mock
private UserRepository
userRepository;
3
Stub Repository Methods
when(userRepository.findById(1L))
.thenReturn(Optional.of(user));
when(userRepository.save(any(User.class)))
.thenReturn(savedUser);
when(userRepository.findAll())
.thenReturn(Arrays.asList(user));
4
Use in Service Under Test
UserService service =
new UserService(userRepository);
User result = service.getUser(1L);
service.createUser(newUser);
5
Verify Interactions
verify(userRepository).findById(1L);
verify(userRepository).save(any(User.class));
verify(userRepository, never())
.deleteById(anyLong());
COMMON REPOSITORY STUBBING
Find By ID
when(repo.findById(1L))
.thenReturn(Optional.of(user));
when(repo.findById(99L)
.thenReturn(Optional.empty());
Find All
when(repo.findAll())
.thenReturn(Arrays.asList(
user1, user2
));
Save
when(repo.save(any(User.class)))
.thenAnswer(invocation -> {
User u = invocation.getArgument(0);
u.setId(1L);
return u;
}));
Delete
doNothing().when(repo)
.deleteById(anyLong());
doThrow(new RuntimeException("DB Error"))
.when(repo).deleteById(anyLong());
existsBy... / Count
when(repo.existsById(1L))
.thenReturn(true);
when(repo.count())
.thenReturn(5L);
BENEFITS OF MOCKING REPOSITORIES
Isolates business logic from database
Faster and reliable unit tests
Control data and scenarios easily
Test edge cases and error paths
Verify interactions with persistence layer

MOCKING EXTERNAL SERVICES

Mock external service calls (payment gateways, email/notifications, third-party APIs) to test without real network calls.

Payment Gateways

- Stripe, PayPal, Braintree. Mock the gateway interface, not the SDK — test approved and declined responses.
- interface PaymentGateway {
-   PaymentResult charge(...); }

Notification Services

- Email, SMS, push. Wrap the vendor client in an app-owned port; mock the port, not the SDK.
- interface CustomerNotificationGateway {
-   void sendActivated(Customer c); }

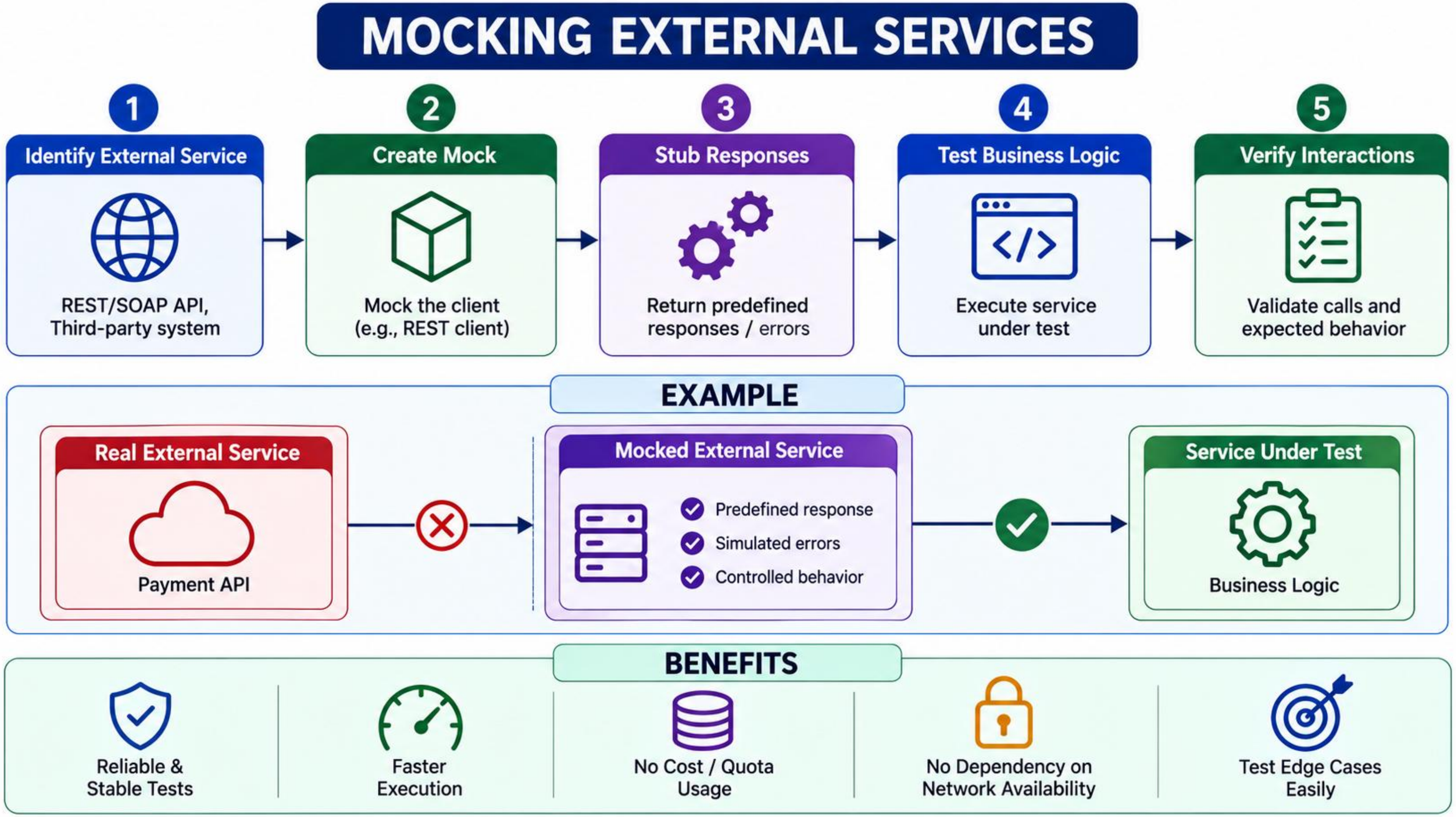
Third-Party APIs

- Partner and internal REST APIs. Mock your own gateway interface, not deep framework chains (e.g., WebClient) — simulate timeouts and 4xx/5xx errors at that boundary.

BEST PRACTICE

- Return success and failure responses to test both the happy path and error-handling logic in your service.
- Blind spot: Use Mockito for application behavior. Use HTTP stubs, contract tests, or integration tests for protocol and payload correctness.

KEY TAKEAWAY Mock the application-owned boundary interface, not the vendor SDK or framework client — keep tests focused on your own logic, not third-party protocols.


MOCKING EXTERNAL SERVICES
1
Identify External Service
REST/SOAP API, Third-party system
2
Create Mock
Mock the client (e.g., REST client)
3
Stub Responses
Return predefined responses / errors
4
Test Business Logic
Execute service under test
5
Verify Interactions
Validate calls and expected behavior
EXAMPLE
Real External Service
Payment API
Mocked External Service
Predefined response
Simulated errors
Controlled behavior
Service Under Test
Business Logic
BENEFITS
Reliable & Stable Tests
Faster Execution
No Cost / Quota Usage
No Dependency on Network Availability
Test Edge Cases Easily

MOCKING SERVICE LAYERS

Mock a service dependency when testing a controller or another service that calls it.

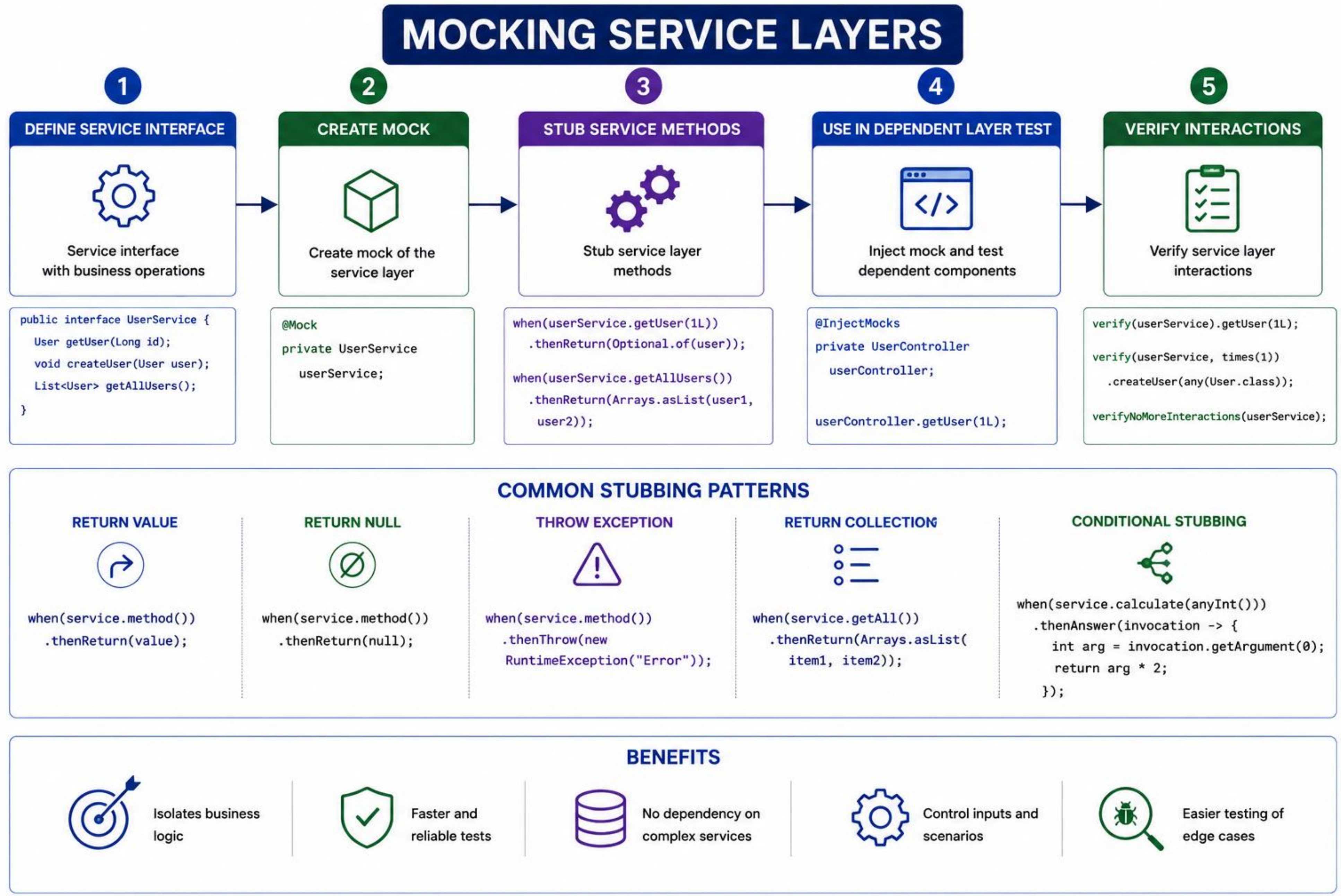
EXAMPLE

```
@Mock
private PricingService pricingService;

when(pricingService.calculateTotal(order))
    .thenReturn(new BigDecimal("199.99"));
```

WHY MOCK A SERVICE LAYER

- Test one layer at a time — the controller, not the service it calls.
- Avoid cascading real logic across multiple layers in a single test.
- Makes it easy to simulate edge cases the dependent service would produce.
- Mock an application service when testing a controller adapter. When testing one service that collaborates with another, consider whether the second collaborator is truly outside the unit or whether a broader component test would provide better confidence.

KEY TAKEAWAY Mocking service layers keeps each test focused on a single unit, even in a multi-layered architecture.


MOCKING SERVICE LAYERS
1
DEFINE SERVICE INTERFACE
Service interface with business operations
public interface UserService {
User getUser(Long id);
void createUser(User user);
List<User> getAllUsers();
}
2
CREATE MOCK
Create mock of the service layer
@Mock
private UserService
userService;
3
STUB SERVICE METHODS
Stub service layer methods
when(userService.getUser(1L))
.thenReturn(Optional.of(user));
when(userService.getAllUsers())
.thenReturn(Arrays.asList(user1,
user2));
4
USE IN DEPENDENT LAYER TEST
Inject mock and test dependent components
@InjectMocks
private UserController
userController;
userController.getUser(1L);
5
VERIFY INTERACTIONS
Verify service layer interactions
verify(userService).getUser(1L);
verify(userService, times(1))
.createUser(any(User.class));
verifyNoMoreInteractions(userService);
COMMON STUBBING PATTERNS
RETURN VALUE
when(service.method())
.thenReturn(value);
RETURN NULL
when(service.method())
.thenReturn(null);
THROW EXCEPTION
when(service.method())
.thenThrow(new
RuntimeException("Error"));
RETURN COLLECTION
when(service.getAll())
.thenReturn(Arrays.asList(
item1, item2));
CONDITIONAL STUBBING
when(service.calculate(anyInt()))
.thenAnswer(invocation -> {
int arg = invocation.getArgument(0);
return arg * 2;
});
BENEFITS
Isolates business logic
Faster and reliable tests
No dependency on complex services
Control inputs and scenarios
Easier testing of edge cases

ISOLATED SERVICE TEST WORKFLOW

A repeatable seven-step workflow for testing business logic against real domain data and mocked boundaries.

1. Arrange real domain data
2. Mock external boundaries
3. Stub only required behavior
4. Call the real service
5. Assert returned state or exception
6. Verify meaningful interactions
7. Confirm forbidden interactions did not occur

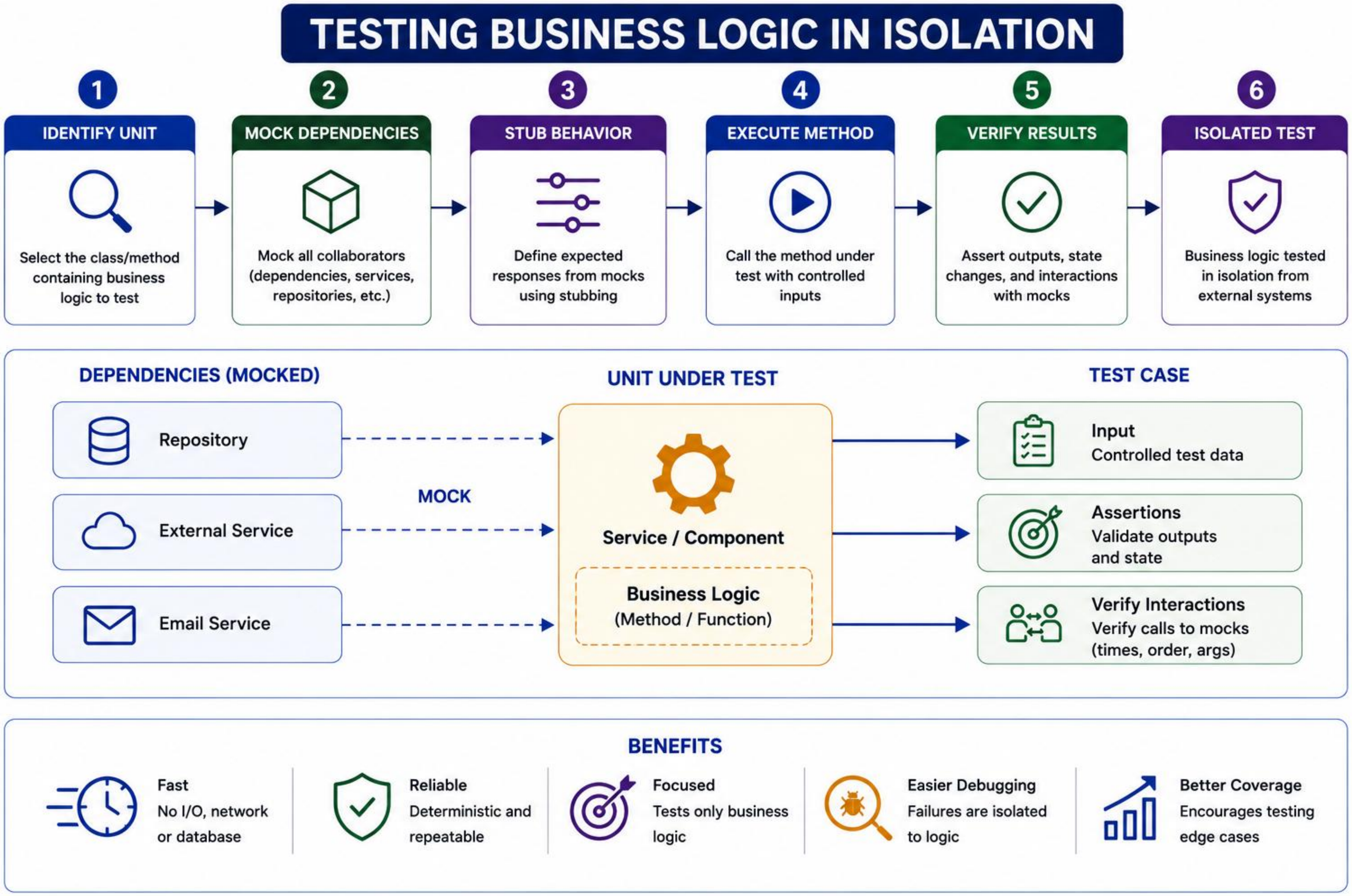
WHAT TO TEST

- Success paths (order placed, payment succeeds) and failure paths (payment declined, item not found) — both matter equally.

BEST PRACTICES

- Follow the AAA pattern: Arrange, Act, Assert.
- Test one business scenario per test.
- Use meaningful test data.
- Keep tests small, focused, and readable.

KEY TAKEAWAY Follow the same seven-step workflow for every isolated service test — from arranging real data to confirming forbidden interactions never happened.


TESTING BUSINESS LOGIC IN ISOLATION
1
IDENTIFY UNIT
Select the class/method containing business logic to test
2
MOCK DEPENDENCIES
Mock all collaborators (dependencies, services, repositories, etc.)
3
STUB BEHAVIOR
Define expected responses from mocks using stubbing
4
EXECUTE METHOD
Call the method under test with controlled inputs
5
VERIFY RESULTS
Assert outputs, state changes, and interactions with mocks
6
ISOLATED TEST
Business logic tested in isolation from external systems
DEPENDENCIES (MOCKED)
Repository
External Service
Email Service
MOCK
UNIT UNDER TEST
Service / Component
Business Logic
(Method / Function)
TEST CASE
Input
Controlled test data
Assertions
Validate outputs and state
Verify Interactions
Verify calls to mocks (times, order, args)
BENEFITS
Fast
No I/O, network or database
Reliable
Deterministic and repeatable
Focused
Tests only business logic
Easier Debugging
Failures are isolated to logic
Better Coverage
Encourages testing edge cases

COMMON MOCKING MISTAKES

Avoid these pitfalls to keep your mocked tests reliable and meaningful.

MISTAKE	PROBLEM	BETTER APPROACH
`Over-mocking`	Tests verify implementation, not behavior	Mock only true external dependencies
`Over-stubbing`	Stubs unused by the test add noise and confusion	Stub only what the test actually needs
`Verifying too much`	Brittle tests break on harmless refactors	Verify only interactions that matter
`Mocking value objects / DTOs`	Unnecessary and misleading	Use real instances for simple data objects
`Ignoring verifyNoMoreInteractions`	Missed unexpected calls go unnoticed	Use it when interaction completeness matters
`Mocking the class under test`	No real behavior is exercised	Instantiate the real class and mock only collaborators

KEY TAKEAWAY Mock dependencies, not the logic you're trying to test — mocking too much hides real bugs.

LAB OVERVIEW – MOCKITO AND MOCKING WITH AI ASSISTANCE

Use Mockito to create mocks, stub methods, verify interactions, and test business logic in isolation.

SCENARIO

- Isolate DefaultCustomerService (Northstar CRM) by mocking CustomerRepository — keep CustomerValidator real so uniqueness rules still run.
- Verify find/save interactions, capture saved entities with ArgumentCaptor, and prove not-found paths never call save.

TECH STACK

ITEM	VALUE
`Language`	Java 21
`Testing`	JUnit 5, Mockito
`Build`	Maven
`IDE`	IntelliJ IDEA

KEY TAKEAWAY By the end of this lab, you'll write effective unit tests using Mockito and apply mocking techniques in real projects.

LAB TASKS AND DELIVERABLES

Complete these tasks to gain hands-on experience with Mockito — each builds on the previous one.

#	TASK	KEY ACTIVITIES	FOCUS AREA
1	Create Mocks	Use @Mock for each dependency; initialize with @ExtendWith(MockitoExtension.class).	Mock setup
2	Stub Methods	Use when().thenReturn()/thenThrow(); stub only what's necessary.	Behavior definition
3	Verify Interactions	Use verify(mock).method(), times(), never(), atLeastOnce().	Interaction verification
4	Mock Repositories	Stub scenario-by-scenario: existing lookup, missing lookup, save — only what the service actually calls.	Data access isolation
5	ArgumentCaptor & BDD	Capture the saved Customer; assert ID, name/email, status, and a single save. Add CustomerServiceBddMockTest using given()/then().should().	Captor + BDD style
6	Prove Isolation	verify(repository, never()).save(any(Customer.class)) and the thrown exception for CUS-9999 (or verifyNoInteractions(repository)). Run individually, as a suite, and reordered.	Negative paths & docs

SUCCESS CRITERIA

- CustomerServiceMockitoTest and CustomerServiceBddMockTest with a mocked CustomerRepository; verified interactions, ArgumentCaptor, and never().save() on not-found; clean, readable test code.

AI ASSISTANCE (OPTIONAL)

- Use GitHub Copilot to draft a mock setup, then review it: reject drafts that mock the class under test, leave unused stubs, or add Thread.sleep. Log: the prompt used, whether the suggestion was accepted or rejected and why, and any hallucinated API or unnecessary stub identified.

KEY TAKEAWAY Apply Mockito to create clean, isolated, reliable unit tests that validate behavior, not implementation.

MOCKITO TEST: DEFINITION OF DONE

Use this checklist to confirm an isolated Mockito test is complete before it ships.

1. Real class under test is instantiated
2. Unit boundary is clear
3. Simple domain objects remain real
4. Only external or controlled collaborators are replaced
5. Stubs are minimal and scenario-specific
6. Observable result is asserted
7. Important interactions are verified
8. Forbidden side effects are verified where relevant
9. Captors are used only when arguments matter
10. No unused stubs remain
11. Test does not depend on invocation order unless order is contractual
12. Repository and network integration are tested elsewhere
13. AI-generated Mockito code has been reviewed

KEY TAKEAWAY Testing in isolation leads to faster, more reliable, and maintainable tests that focus on behavior, not implementation.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of Mockito and test isolation.

1. A test mocks Customer, CustomerValidator, CustomerRepository, and DefaultCustomerService itself. What is the primary problem?

- **A.** Missing @ExtendWith(MockitoExtension.class)
- **B. The mocked class means no real logic runs.**
- **C.** Too many dependencies are mocked at once
- **D.** CustomerValidator should not be mocked

2. A service returns the correct result, but the test verifies six internal repository calls not part of its public behavior. What risk does this create?

- **A.** The test will fail intermittently
- **B. A brittle, implementation-coupled test.**
- **C.** The test runs slower than necessary
- **D.** Mockito throws an UnnecessaryStubbingException

3. What does @InjectMocks do?

- **A.** Creates a mock for the class under test
- **B. Injects created mocks into the real object under test**
- **C.** Verifies method calls
- **D.** Starts a database connection

4. Best way to verify a method was never called?

- **A. verify(mock, never()).method()**
- **B.** verify(mock, times(0))
- **C.** assertNull(mock)
- **D.** when(mock).never()

KEY TAKEAWAY Mockito's @Mock, when()/thenReturn(), and verify() form the core toolkit for isolated unit testing.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. What is a common mocking mistake?

A. Stubbing only what's needed
B. Mocking every dependency including value objects
C. Verifying important interactions
D. Using meaningful test names

6. Why mock external services like payment gateways?

A. To slow down the test suite
B. To avoid real network calls and simulate failures easily
C. To make tests depend on live systems
D. To replace unit tests entirely

7. What should you verify in a well-designed test?

A. Every internal method call
B. Only interactions that matter to the test's purpose
C. Private field values
D. Nothing — verification is optional

8. What is the main benefit of testing in isolation?

A. Tests run against production data
B. Fast, deterministic tests that pinpoint real bugs
C. Tests require a live database
D. Tests take longer but are more thorough

ANSWER KEY

- **1-B 2-B 3-B 4-A 5-B 6-B 7-B 8-B**

KEY TAKEAWAY Mockito enables test isolation by mocking dependencies — the result is faster, more reliable, more maintainable tests.

MODULE 18 COMPLETE

Mockito for Test Isolation

You can now use Mockito confidently to write clean, isolated unit tests that validate behavior, not implementation.